



南开大学
Nankai University

南 开 大 学

数 学 科 学 学 院

深度学习原理实验报告

MNIST 数据集手写数字识别

谢 博

年级：2024 级

专业：数学与人工智能

指导教师：刘晓滨

2026 年 6 月 7 日

摘要

本实验基于 PyTorch 构建并训练一个卷积神经网络, 用于完成 MNIST 手写数字识别任务。实验围绕数据加载与预处理、网络结构设计、模型训练、测试评估和结果可视化展开。模型输入为 $1 \times 28 \times 28$ 的灰度图像, 主体结构包含两层卷积层和两层全连接层, 并使用 ReLU 激活函数、最大池化和 log softmax 输出完成多分类建模。训练阶段采用 Adam 优化器和负对数似然损失函数, 在 20 个 epoch 后, 模型测试准确率达到 99.11%。实验结果表明, 卷积神经网络能够有效提取手写数字图像中的局部空间特征, 并在 MNIST 分类任务上取得较好的识别效果。本次作业相关代码已上传至 github, 仓库地址为 <https://github.com/Zemiss/MnistCNN-Coursework>

关键字: MNIST; 卷积神经网络; PyTorch; 手写数字识别; 深度学习

目录

一、 实验目的	1
二、 实验原理	1
(一) MNIST 数据集	1
(二) 卷积神经网络原理	1
(三) 损失函数与评价指标	1
三、 实验步骤	2
(一) 环境配置	2
(二) 项目组织	2
(三) 训练流程	3
(四) 测试流程	3
四、 程序代码	3
(一) 模型定义代码	3
(二) 训练代码	4
(三) 测试评价代码	4
五、 实验结果显示	5
(一) 真实标签样例	5
(二) 训练指标	5
(三) 损失曲线	6
(四) 准确率曲线	6
(五) 预测样例	7
六、 实验分析总结	7

一、实验目的

本实验使用 PyTorch 构建卷积神经网络，对 MNIST 手写数字图像进行分类识别。实验目标是完成从数据加载、模型定义、训练优化、测试评估到结果可视化的完整流程，并观察损失函数和准确率随训练轮数变化的趋势。

通过本实验，可以理解卷积层、池化层、全连接层和激活函数在图像分类中的作用，掌握使用 `DataLoader` 组织图像数据、使用 Adam 优化器训练神经网络、使用测试集准确率评估模型泛化能力的方法。同时，本实验也训练了将实验代码、指标文件和可视化结果整理为规范实验报告的能力。

二、实验原理

(一) MNIST 数据集

MNIST 是经典的手写数字识别数据集，包含数字 0 到 9 共 10 个类别。每张样本图像为单通道灰度图，尺寸为 28×28 。本项目使用 `torchvision.datasets.MNIST` 自动加载数据，其中训练集包含 60000 张图像，测试集包含 10000 张图像。

在输入模型前，图像首先通过 `transforms.ToTensor()` 转换为张量，并将像素值缩放到 0 到 1。随后使用 `transforms.Normalize((0.1307,), (0.3081,))` 按 MNIST 数据集的均值和标准差进行标准化。标准化可以使输入分布更加稳定，有助于模型训练。

(二) 卷积神经网络原理

卷积神经网络适合处理图像数据。卷积层通过局部连接和权值共享提取图像中的笔画、边缘和局部结构特征。池化层通过下采样降低特征图尺寸，并增强模型对局部位置变化的鲁棒性。全连接层则将卷积层提取到的特征映射到具体类别。

本实验采用的模型为 ConvNet。模型输入为 $1 \times 28 \times 28$ 的灰度图像，输出为 10 个类别的对数概率。网络包含两层卷积层和两层全连接层，结构如表 1 所示。

层次	操作	输出形状
输入	MNIST 灰度图像	$1 \times 28 \times 28$
Conv1	5×5 卷积, 10 个通道, ReLU, 2×2 最大池化	$10 \times 12 \times 12$
Conv2	3×3 卷积, 20 个通道, ReLU	$20 \times 10 \times 10$
Flatten	展平特征图	2000
FC1	全连接层, ReLU	500
FC2	全连接层	10
输出	log softmax	10

表 1: 卷积神经网络结构

(三) 损失函数与评价指标

模型最后一层使用 `log_softmax` 输出各类别的对数概率，因此训练时采用负对数似然损失函数 `F.nll_loss`。对于单个样本，若真实类别为 y ，模型对该类别输出的对数概率为 $\log p_y$ ，则

损失为：

$$L = -\log p_y \quad (1)$$

模型分类性能使用准确率衡量。准确率定义为预测正确样本数与总样本数的比值：

$$Accuracy = \frac{N_{correct}}{N_{total}} \quad (2)$$

三、 实验步骤

（一） 环境配置

本项目使用 Python 3.11 和 PyTorch 实现。主要依赖包括 torch、torchvision、pandas、matplotlib、Pillow 和 PyYAML。环境配置命令如下：

环境配置命令

```
1 conda create -n mnist-cnn python=3.11 -y
2 pip install -r requirements.txt
```

（二） 项目组织

项目中，训练脚本负责模型训练、指标记录和图像保存，测试脚本负责加载模型并执行预测。模型文件定义卷积神经网络，工具文件封装训练、测试和绘图函数，配置文件保存默认参数。主要目录结构如下：

项目目录结构

```
1 .
2 |-- configs/
3 |   |-- default.yaml
4 |-- src/
5 |   |-- config.py
6 |   |-- model.py
7 |   |-- utils.py
8 |-- train.py
9 |-- test.py
10 |-- requirements.txt
11 |-- model/
12 |   |-- mnist_cnn.pth
13 |-- outputs/
14 |   |-- metrics.csv
15 |   |-- loss_curve.png
16 |   |-- accuracy_curve.png
17 |   |-- sample_ground_truth.png
18 |   |-- sample_predictions.png
```

(三) 训练流程

实验默认批大小为 512，训练轮数为 20，随机种子为 1，优化器为 Adam，学习率为 0.001。训练脚本会优先使用 CUDA 设备；如果没有可用 GPU，则自动使用 CPU。每个 batch 中，脚本依次执行前向传播、损失计算、反向传播和参数更新。每个 epoch 结束后，脚本在测试集上计算平均损失和准确率。

模型保存策略为最佳测试准确率保存。只有当当前 epoch 的测试准确率高于历史最佳值时，脚本才会将权重保存到 `model/mnist_cnn.pth`。训练结束后，脚本会保存 `metrics.csv`、损失曲线、准确率曲线、真实标签样例和预测样例。

训练命令如下：

训练命令

```
1 python train.py
```

(四) 测试流程

训练完成后，可以使用 `test.py` 加载保存的模型权重进行测试。若直接使用 MNIST 测试集，可执行如下命令：

使用 MNIST 测试集测试

```
1 python test.py --test-data-dir data --model-path model\mnist_cnn.pth --use-mnist --num-samples 20
```

测试脚本也支持读取自定义图片文件夹，并逐张输出预测类别和对应概率。支持的图片格式包括 `.png`、`.jpg`、`.jpeg` 和 `.bmp`。使用自定义图片文件夹时，可执行如下命令：

使用图片文件夹测试

```
1 python test.py --test-data-dir path\to\digit_images --model-path model\mnist_cnn.pth --no-use-mnist
```

四、 程序代码

(一) 模型定义代码

核心网络定义位于 `src/model.py`。模型先通过两层卷积提取图像特征，再将特征展平并输入两层全连接层，最后输出 10 个类别的对数概率。

ConvNet 模型定义

```
1 class ConvNet(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.conv1 = nn.Conv2d(1, 10, 5)
5         self.conv2 = nn.Conv2d(10, 20, 3)
6         self.fc1 = nn.Linear(20 * 10 * 10, 500)
7         self.fc2 = nn.Linear(500, 10)
8
9     def forward(self, x):
10         in_size = x.size(0)
```

```
11 out = self.conv1(x)
12 out = F.relu(out)
13 out = F.max_pool2d(out, 2, 2)
14 out = self.conv2(out)
15 out = F.relu(out)
16 out = out.view(in_size, -1)
17 out = self.fc1(out)
18 out = F.relu(out)
19 out = self.fc2(out)
20 return F.log_softmax(out, dim=1)
```

(二) 训练代码

训练过程封装在 `src/utils.py` 的 `train_epoch` 函数中。每个 batch 先清空梯度，再计算模型输出和损失，随后执行反向传播和优化器更新。

单轮训练核心代码

```
1 for batch_idx, (data, target) in enumerate(train_loader):
2     data = data.to(device)
3     target = target.to(device)
4
5     optimizer.zero_grad()
6     output = model(data)
7     loss = F.nll_loss(output, target)
8     loss.backward()
9     optimizer.step()
10
11     pred = output.data.max(1, keepdim=True)[1]
12     correct += pred.eq(target.data.view_as(pred)).sum().item()
```

(三) 测试评价代码

测试过程在不计算梯度的条件下执行。脚本将模型切换到评价模式，累计测试集损失和预测正确样本数，并计算测试准确率。

测试评价核心代码

```
1 model.eval()
2 test_loss = 0.0
3 correct = 0
4
5 for data, target in test_loader:
6     data = data.to(device)
7     target = target.to(device)
8     output = model(data)
9     test_loss += F.nll_loss(output, target, reduction="sum").item()
10    pred = output.data.max(1, keepdim=True)[1]
11    correct += pred.eq(target.data.view_as(pred)).sum().item()
```

```

12
13 test_loss /= len(test_loader.dataset)
14 test_acc = correct / len(test_loader.dataset)

```

五、 实验结果显示

(一) 真实标签样例

图 1 展示了测试集中部分手写数字样例及其真实标签。可以看到，MNIST 图像均为灰度图，数字形态存在一定差异，这要求模型能够学习具有鲁棒性的局部笔画特征。

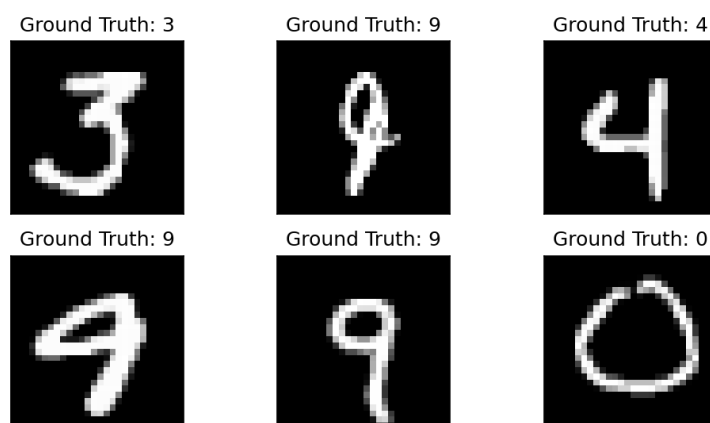


图 1: MNIST 测试样例真实标签

(二) 训练指标

本次训练共运行 20 个 epoch。训练损失从第 1 轮的 0.3273 降至第 20 轮的 0.0011，训练准确率从 90.91% 提升到 99.98%。测试损失从第 1 轮的 0.1155 降至第 20 轮的 0.0379，测试准确率从 96.33% 提升到 99.11%。最高测试准确率出现在第 20 轮，为 99.11%。

Epoch	Train Loss	Train Acc	Test Loss	Test Acc
1	0.3273	90.91%	0.1155	96.33%
5	0.0340	98.92%	0.0395	98.73%
10	0.0084	99.76%	0.0374	98.92%
15	0.0037	99.90%	0.0375	99.10%
20	0.0011	99.98%	0.0379	99.11%

表 2: 部分 epoch 的训练与测试指标

(三) 损失曲线

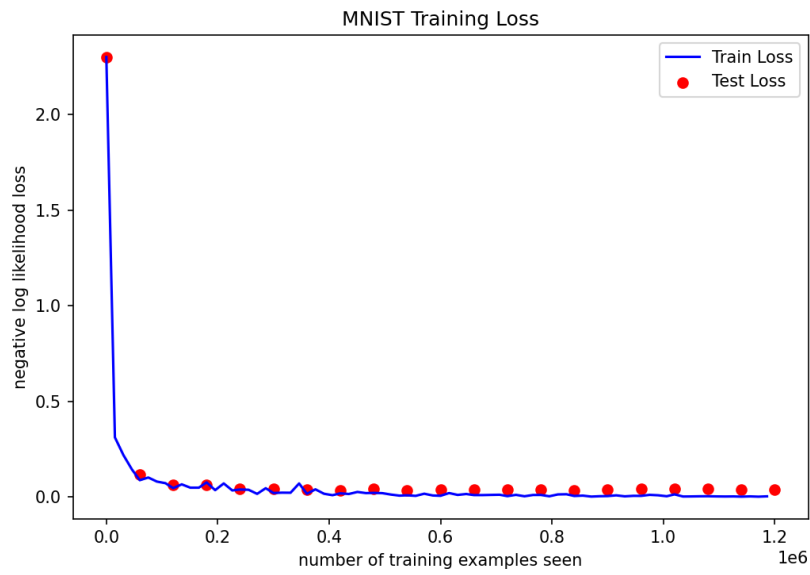


图 2: 训练损失与测试损失变化曲线

(四) 准确率曲线

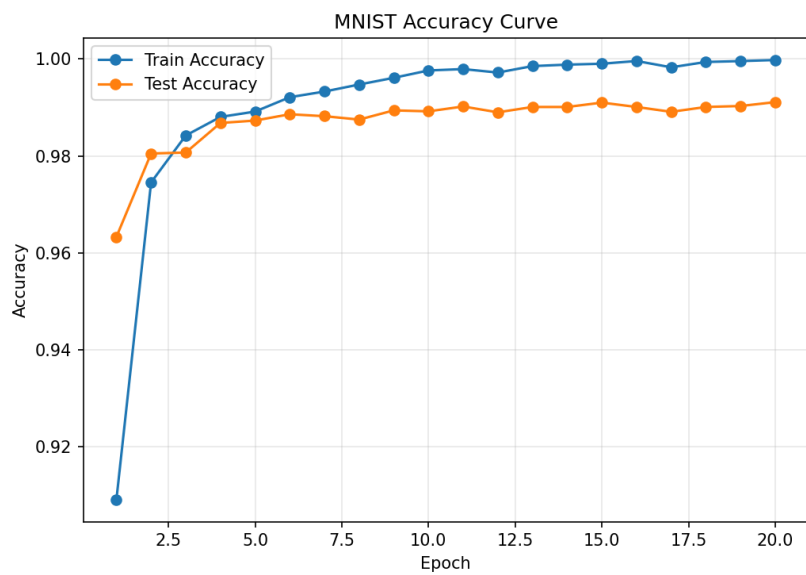


图 3: 训练准确率与测试准确率变化曲线

(五) 预测样例

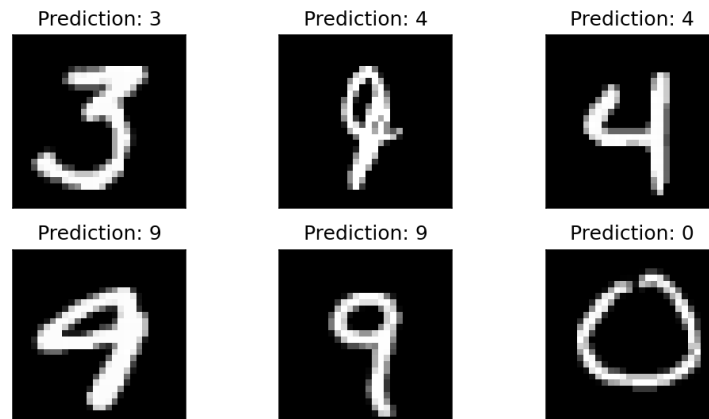


图 4: MNIST 测试样例预测结果

六、 实验分析总结

从表 2 和图 2 可以看出，模型在前几个 epoch 内快速收敛。第 5 轮时训练准确率已经达到 98.92%，测试准确率达到 98.73%。这说明卷积网络能够较快学习 MNIST 图像中的主要判别特征。随着训练继续进行，训练损失持续下降，训练准确率逐步接近 100%。

图 3 显示，测试准确率在第 6 轮后基本稳定在 98.8% 以上，并在后续训练中缓慢提升到 99.11%。训练准确率和测试准确率之间存在一定差距，这是模型在训练样本和未见样本上表现不同造成的正常泛化误差。测试损失在训练后期存在轻微波动，说明模型已经接近收敛，继续训练对测试准确率的提升有限。

图 4 展示了模型在测试样例上的预测结果。训练后的卷积神经网络能够识别不同形态的手写数字，并给出与图像内容一致的类别预测。整体来看，两层卷积层和两层全连接层构成的 CNN 已经能够较好地完成 MNIST 手写数字分类任务。

本实验的不足在于模型结构较简单，未加入 dropout、数据增强或学习率调度等改进手段。若进一步扩展实验，可以比较不同优化器、学习率、batch size 和网络深度对测试准确率的影响，也可以在自定义手写数字图片上测试模型的实际泛化能力。